From Prompt to Exploit: Security Vulnerabilities in LLM-Generated Smart Contracts

CSC 5991 — Introduction to LLM: Project Proposal

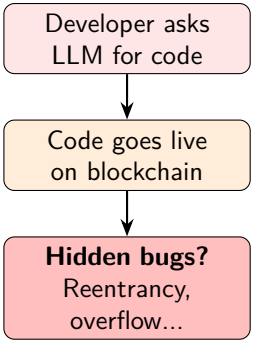
Anonymous Author(s)

Anonymous Institution

March 2026

Why This Matters

- Developers today often use tools like ChatGPT and Copilot to write smart contracts in Solidity
- These contracts handle real money — sometimes billions of dollars in DeFi protocols
- The catch: once you deploy a smart contract, you **can't patch it**. Bugs stay forever
- So we wanted to ask a simple question: **how safe is the code that LLMs actually produce?**



What We Want to Find Out

RQ1. How common are security bugs in contracts that LLMs generate?

RQ2. Are some LLMs better or worse than others at writing safe code?

RQ3. What kinds of bugs show up the most?

RQ4. Can we trick LLMs into writing even more vulnerable code with sneaky prompts?

RQ5. Does the type of contract matter? (e.g., a simple token vs. a complex bridge)

RQ6. Are there any bug patterns that are unique to LLM-written code?

Our Approach

1. **Design prompts** — we wrote 300 prompts covering 20 types of smart contracts (tokens, DeFi, bridges, governance, etc.)
2. **Generate contracts** — we feed those prompts to 8 different LLMs and collect the Solidity code they produce (2,400 contracts total)
3. **Scan for vulnerabilities** — we run three industry-standard security tools (Slither, Mythril, Semgrep) on every contract
4. **Test adversarial prompts** — we also try prompts designed to trick the LLM into writing unsafe code, and compare
5. **Statistical analysis** — we use non-parametric

8 LLMs we are testing:

- GPT-4o (OpenAI)
- GPT-5 (OpenAI)
- Claude 3.5 Sonnet (Anthropic)
- Gemini 2.5 Flash-Lite (Google)
- DeepSeek Coder V2
- Qwen 2.5 Coder 32B
- CodeLlama-34B (Meta)
- Codestral 22B (Mistral)

3 security tools:

- Slither — fast static analysis
- Mythril — symbolic execution
- Semgrep — pattern-based rules

What We Have Done So Far

Work completed:

- ✓ Generated all 2,400 contracts from 8 LLMs
- ✓ Ran Slither on everything — 1,566 compiled successfully, found 4,342 issues
- ✓ Ran Semgrep on all 2,400 — picked up 1,296 more issues
- ✓ Sampled 499 contracts for Mythril (deeper analysis) — 262 issues found
- ✓ After removing duplicates: **5,134 total bugs**
- ✓ All statistical tests done

What we found (so far):

- About **2 out of 3** compilable contracts have at least one vulnerability (64.5%)
- Roughly **26.8%** have high-severity bugs
- Some LLMs can't even produce valid code — compilation rates range from 34.7% to 91.0%
- There are big differences between LLMs (statistically significant)
- Surprisingly, adversarial prompts did **not** make things worse

Comparing the LLMs

Model	Type	Compiles	% Vulnerable	Avg Bugs	Bugs/100 LOC	Avg LO
GPT-4o	General	90.0%	73.0%	3.05	6.61	52
GPT-5	General	**91.0%**	35.5%	1.79	**2.22**	96
Claude 3.5 Sonnet	General	68.3%	84.9%	7.64	6.98	113
Gemini 2.5 Flash-Lite	General	49.7%	80.5%	4.52	**8.43**	55
Qwen 2.5 Coder 32B	Code	79.0%	66.2%	2.75	6.95	42
DeepSeek Coder V2	Code	68.3%	73.2%	2.98	7.96	38
CodeLlama-34B	Code	34.7%	49.0%	1.23	5.16	26
Codestral 22B	Code	41.0%	52.0%	1.57	6.88	24

What this tells us

- GPT-5 compiles the best (91%) and has the lowest bug density (2.22/100 LOC)
- Claude writes the biggest contracts (113 LOC avg), so it ends up with the most total bugs
- Gemini has the worst bug density — 8.43 bugs per 100 lines of code

How Does LLM Code Compare to Human Code?

Metric	Human (Etherscan)	LLM (8 models)
Total contracts	241	2,400
Compiled	132 (54.8%)	1,566 (65.2%)
% with vulnerabilities	82.6%	64.5%
Mean findings/contract	11.51	3.28
Mean LOC	446.7	57.2
Vulns/100 LOC	**2.69**	**2.22–8.43**
High severity %	16.0%	15.7%

The real story

- Per line of code, vulnerability density is **comparable** (2.69 vs 2.22–8.43)
- Both trigger the **same** detectors: reentrancy-events, timestamp, missing-zero-check
- High-severity proportion nearly identical (16.0% vs 15.7%)
- The danger is not worse code — it's **skipping the security pipeline** (audits, monitoring, testing) that makes human code survivable

Project Timeline

When	What we did / plan to do	Status
Jan 2026	Designed 300 prompts across 20 contract types	Done
Jan–Feb	Generated 2,400 contracts from 8 LLMs	Done
Feb	Ran Slither and Semgrep on all contracts	Done
Feb–Mar	Ran Mythril on a stratified sample of 499 contracts	Done
Mar (week 1)	Aggregated results, removed duplicates	Done
Mar (week 2)	Statistical analysis and adversarial comparison	Done
Mar (week 3)	Human baseline: 241 Etherscan contracts analyzed	Done
Mar (week 4)	Manually review 120 contracts for false positives	To do
Mar–Apr	Root cause analysis — why LLMs produce these bugs	To do
Apr	Test prompt-based mitigation strategies	To do

What's still left

- **Manually validate** a sample of 120 contracts to confirm tool findings
- **Root cause analysis** — investigate *why* LLMs produce these vulnerabilities (e.g., training data

What We Hope to Contribute

1. **A large-scale study** — 2,400 contracts across 8 LLMs and 20 categories; as far as we know, nobody has done this at this scale before
2. **A clear taxonomy of bugs** — we map every vulnerability to a standard CWE identifier so people can compare results
3. **A surprising result** — adversarial prompts don't actually make LLMs write worse code; the bugs seem to come from the training data itself
4. **An open dataset** — we plan to release all contracts, prompts, and analysis scripts so anyone can build on this work
5. **Practical takeaways** — concrete advice for developers who use LLMs for smart contract development

Thank you! Questions?